University of Moratuwa

Faculty of Information Technology

SwiftDrop

Cloud-Native Parcel Delivery & Courier Management Platform

Technical Project Report

Module

Enterprise Application Development (IN3101)

Group 4

Project Repository

`https://github.com/Kevin200307/enterpriseProject.git`

Declaration and Team Composition

This report documents the design and implementation of **SwiftDrop**, a distributed parcel delivery and courier management platform developed by a ten-member engineering team. Every architectural claim, diagram, table and figure in this document has been verified directly against the project source code.

Index No.	Member	Area of Ownership	GitHub
234021D	Arangala A. P.	Identity & Authentication	`@AmandiArangala`
	Kevin Gaveshana	API Gateway & Edge Platform	`@Kevin200307`
234127H	M.A.P.M. Mallawasekara	Pricing & Zone Management	`@Pavindu11`
234087H	L. P. Ishadi	Parcel Booking & Lifecycle	`@LP-Ishadi`
234149C	Methsara	Dispatch & Courier Workload	`@Dimeshmethsara`
	Charithma Dassanayake	Merchant API & Webhooks	`@CDDassanayake`
234009A	Maheesha Paboth	Live Tracking & WebSockets	`@MaheeshaPaboth`
234208H	Thranjaya S. J. K.	Notification Pipeline	`@thirsturn`
234234H	R.K.T.P. Wijerathna	AI Assistant & Tooling	`@Prabhashwara2003`
234035A	K.G.T. Buddhima	Frontend SPA & System Design	`@ThanojBuddhima`

Contents

Declaration and Team Composition **1**

1 Introduction and Project Context **4**
1.1 Background . . . 4
1.2 Problem Statement . . . 4
1.3 Objectives . . . 4
1.3.1 Functional Objectives . . . 4
1.3.2 AI Assistant Objectives . . . 5
1.3.3 Architectural Objectives . . . 5
1.3.4 Security and Quality Objectives . . . 5
1.4 Technology Stack . . . 6

2 Requirements and Use Case Modelling **7**
2.1 Actors and Authorisation Boundaries . . . 7
2.2 Use Case Specifications . . . 7
2.3 Use Case Diagrams . . . 8

3 System Architecture **13**
3.1 Architectural Style and Justification . . . 13
3.1.1 Honest Cost of This Choice . . . 13
3.2 Layered Topology . . . 14
3.3 Service Inventory . . . 16

4 Database and Entity-Relationship Modelling **18**
4.1 Database-per-Service . . . 18
4.2 The 24-Table Domain Model . . . 18
4.3 Entity-Relationship Diagrams . . . 19
4.3.1 Known Model Drift . . . 22

5 Integration Architecture and Messaging **23**
5.1 Integration Patterns . . . 23
5.2 Messaging Topology and Delivery Guarantees . . . 24

6 Core Workflows and State Transitions **27**
6.1 Customer Booking and Dynamic Pricing . . . 27
6.2 Merchant Order Creation via the REST API . . . 28
6.3 Courier Status Update Triggering Notifications . . . 29
6.4 Google OAuth 2.0 Login with PKCE . . . 30
6.5 Parcel Lifecycle State Machine . . . 32

7 AI Assistant Service **34**
7.1 Architecture . . . 34
7.2 Implemented Tools . . . 34
7.3 Designed but Not Yet Implemented . . . 34
7.4 Security of the Tool Path . . . 35

8 Design Patterns and Engineering Principles **36**
8.1 Strategy Pattern in the Pricing Engine 36
8.2 SOLID Principles 36
8.3 Further Patterns Applied 37
8.4 Don't Repeat Yourself 38

9 Security Architecture **39**
9.1 Cryptographic Authentication 39
9.2 Defence in Depth 39

10 Quality Assurance and Defect Transparency **41**
10.1 Testing Strategy 41
10.2 Defect Audit 41
10.3 Consolidated Implementation Status 42

11 Team Contributions and Project Management **43**
11.1 Work Breakdown 43
11.2 Milestone Progression 44

12 Conclusion and Roadmap **45**
12.1 Conclusion 45
12.2 Roadmap 45

Chapter 1

Introduction and Project Context

1.1 Background

The rapid growth of digital commerce has made agile, reliable last-mile logistics essential to modern trade. Traditional courier operations still rely on disconnected records, manual dispatch ledgers and monolithic backends that create operational silos, offer no live visibility, and degrade under peak demand.

SwiftDrop is a cloud-native, distributed parcel delivery and courier management platform that unifies parcel booking, zone-based pricing, courier dispatch, real-time tracking, multi-channel notification and commercial merchant integration. The platform additionally includes an **AI assistant service** built on Spring AI, which exposes read-oriented domain tools to a large language model so users can query the system in natural language.

1.2 Problem Statement

Contemporary logistics platforms exhibit several recurring deficiencies, each of which shaped a design decision in SwiftDrop.

1. **Opaque transit visibility.** Batch-updated records do not reflect instantaneous transit checkpoints. Customers experience uncertainty about delivery windows, and couriers lack a standardised mechanism to capture tamper-evident proof of delivery at the doorstep.
2. **Inflexible, manual pricing.** Coarse rate cards fail to accommodate multi-tier geographic zones, parcel size bands, weight allowances and express service tiers, causing revenue leakage for operators and unexpected charges for customers.
3. **Sub-optimal dispatch and workload imbalance.** Ad-hoc manual assignment leaves dispatchers without unified visibility of courier capacity, producing delayed pickups, uneven workload and elevated failure rates.
4. **Friction in B2B partner integration.** Merchants needing programmatic, high-volume shipping are hindered by proprietary APIs lacking self-service key provisioning, scoped authorisation and resilient webhook pipelines.
5. **Monolithic fragility.** Shared-database contention means high-volume read traffic (thousands of concurrent tracking lookups) degrades critical transactional paths, and a failure in an auxiliary subsystem can cascade across the whole application.
6. **Manual operational overhead.** Routine enquiries - tracking lookups, fee quotations, delivery history - consume human support capacity that a natural-language interface can absorb.

1.3 Objectives

The SwiftDrop platform was designed around four dimensions: functional capability, AI assistant capability, architectural standards, and security and quality assurance.

1.3.1 Functional Objectives

- Provide a self service booking portal that calculates shipping tariffs in real time from the origin and destination zones, parcel size band, weight and service speed.

- Extend tariff calculation to volumetric parcel dimensions and negotiated merchant volume discounts.
- Deliver dedicated, responsive web portals for Customers, Merchants, Couriers, Dispatchers and Administrators.
- Implement push based shipment tracking using STOMP over WebSockets, with Server Sent Events and polling as progressive fallbacks.
- Guarantee asynchronous transactional notification on every parcel lifecycle milestone, addressed to both the sender and the recipient.
- Support email and SMS as independently configurable notification channels governed by per user preferences.
- Supply a secured partner REST API with scoped API keys and outbound HMAC signed webhooks backed by exponential backoff retry.

1.3.2 AI Assistant Objectives

- Implement a tool calling assistant over the domain services using Spring AI, so the platform can be queried in natural language.
- Expose conversational fee estimation, parcel tracking and delivery history as callable tools.
- Extend the assistant with write capable tools for parcel booking, dispatch allocation and merchant operations such as key provisioning and webhook retry.
- Publish the same tool surface over the Model Context Protocol so external clients can consume it.
- Stream the assistant's tool invocations to the client over Server Sent Events for full operational transparency.
- Filter the active tool set at runtime against the caller's granted authorities so the assistant never exceeds the user's own permissions.

1.3.3 Architectural Objectives

- Partition the platform into autonomous Spring Boot microservices aligned with Domain Driven Design bounded contexts.
- Enforce database per service encapsulation, eliminating cross service joins and shared database locks.
- Deploy a reactive Spring Cloud Gateway as the single perimeter ingress, responsible for routing, CORS, token validation, rate limiting and circuit breaking.
- Decouple domain state changes through a RabbitMQ topic exchange to achieve fault tolerance and eventual consistency across downstream consumers.
- Containerise every service and backing dependency, orchestrated locally through Docker Compose.
- Orchestrate the platform on Kubernetes through an umbrella Helm chart covering deployments, services, config maps, secrets and ingress.
- Scale the edge and tracking workloads independently using Kubernetes Horizontal Pod Autoscalers.

1.3.4 Security and Quality Objectives

- Implement RSA 2048 bit RS256 JSON Web Tokens with RFC 7517 JWKS discovery, enabling stateless downstream validation.
- Integrate Google OAuth 2.0 with Proof Key for Code Exchange for federated sign in.

- Rotate refresh tokens on every use and revoke the entire token family on replay detection.
- Verify core business rules and state machines through unit testing with JUnit 5 and Mockito.
- Exercise repository and messaging behaviour against real PostgreSQL and RabbitMQ instances using Testcontainers.
- Assert the role based authorisation matrix so unauthorised roles are provably refused.
- Cover the critical user journeys with end to end browser smoke tests.

1.4 Technology Stack

Layer	**Technology**
Frontend application	React 18.3, TypeScript, Vite, Tailwind CSS, TanStack React Query, React Router
Edge gateway / BFF	Spring Cloud Gateway (Project Reactor, Netty), Redis token-bucket rate limiting, Resilience4j circuit breakers
Backend microservices	Java 21, Spring Boot 4.1.1, Spring Security, Spring Data JPA, Jakarta Bean Validation
Persistence	PostgreSQL on Neon Cloud - *one database per service* (six databases), schema-managed by Flyway with `ddl-auto: validate`
Cache and projection	Redis 7 - rate-limit buckets, login throttling, and the entire tracking read projection
Asynchronous broker	RabbitMQ 3 - topic exchange `parcel.events`, dead-letter exchange `swiftdrop.dlx`
Object storage	MinIO (S3-compatible) - proof-of-delivery photographs. PDF receipts are generated on demand and are not stored
Mail transport	Mailpit for local development; SMTP in deployed environments
AI assistant	Spring AI with an OpenAI-compatible model provider
Local orchestration	Docker Compose (all services plus backing infrastructure)
Kubernetes orchestration	Umbrella Helm chart covering deployments, services, config maps, secrets and ingress

Table 1.1: *Verified technology stack*

Chapter 2

Requirements and Use Case Modelling

2.1 Actors and Authorisation Boundaries

Authorisation is role-based. The five roles are declared once in a shared enumeration (`contracts.domain.Role`) and carried in the `roles` claim of every access token, so the actors below are the system's real authorisation boundaries rather than an analyst's abstraction.

Actor	Role	Capability boundary
Guest	-	Rate-limited public tracking lookup only; the sole capability available without authentication
Customer	`CUSTOMER`	Estimate fees, book deliveries, track parcels, download receipts, manage notification preferences
Merchant	`MERCHANT`	Granted only after administrator approval. Console access, API key issuance, order placement, webhook configuration
Courier	`COURIER`	Inspect assigned manifest, execute guarded status transitions, capture photographic proof, record failure reasons
Dispatcher	`DISPATCHER`	Dispatch queue, courier workload, assignment and unassignment, courier roster management
Administrator	`ADMIN`	Generalises Dispatcher; adds merchant approval, user and role administration, zone and pricing configuration, notification audit

Table 2.1: *System actors and authorisation boundaries*

A merchant's *backend system* is a second, non-human actor that authenticates with a scoped API key rather than a user session.

2.2 Use Case Specifications

Customer
: Register; sign in with password or Google; estimate fee; book delivery; view deliveries; track parcel; download receipt; cancel delivery; manage notification preferences; apply to become a merchant.

Merchant
: Register organisation; await approval; create order through the console; create order through the REST API; list and inspect orders; issue, list and revoke API keys; configure webhook endpoint; inspect and replay webhook deliveries.

Courier
: View daily manifest; mark picked up, in transit and out for delivery; upload proof of delivery; mark delivered; record failure with mandatory reason; reattempt a failed delivery.

Dispatcher / Admin

Monitor dispatch queue; inspect courier workload; request assignment suggestion; assign and unassign parcels; manage courier roster. Administrators additionally approve merchants, manage users and roles, configure zones and pricing, and audit notifications.

2.3 Use Case Diagrams

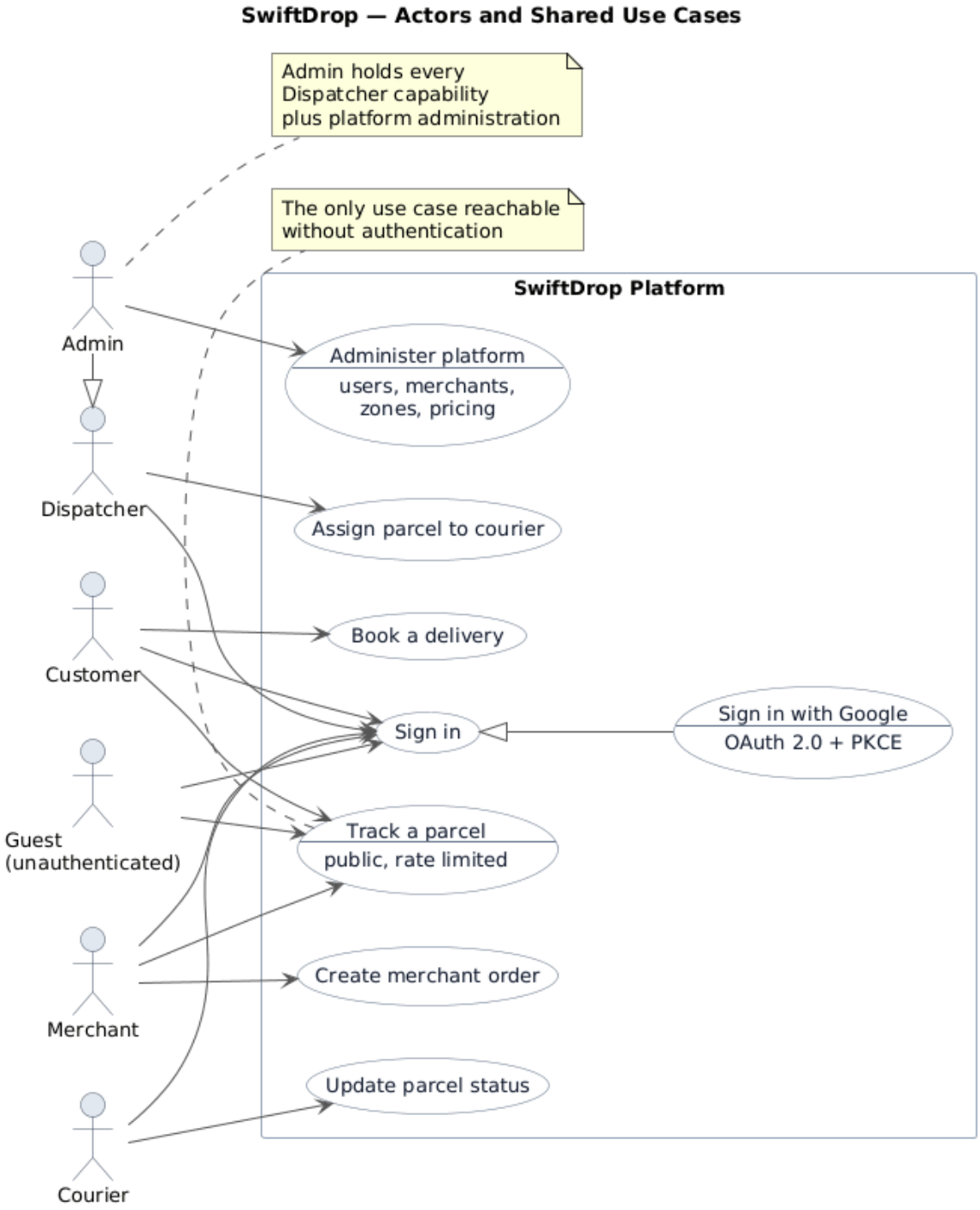


Figure 2.1: *System-level use case overview showing all actors, the generalisation of* `DISPATCHER` *by* `ADMIN`*, and the capabilities shared across roles*

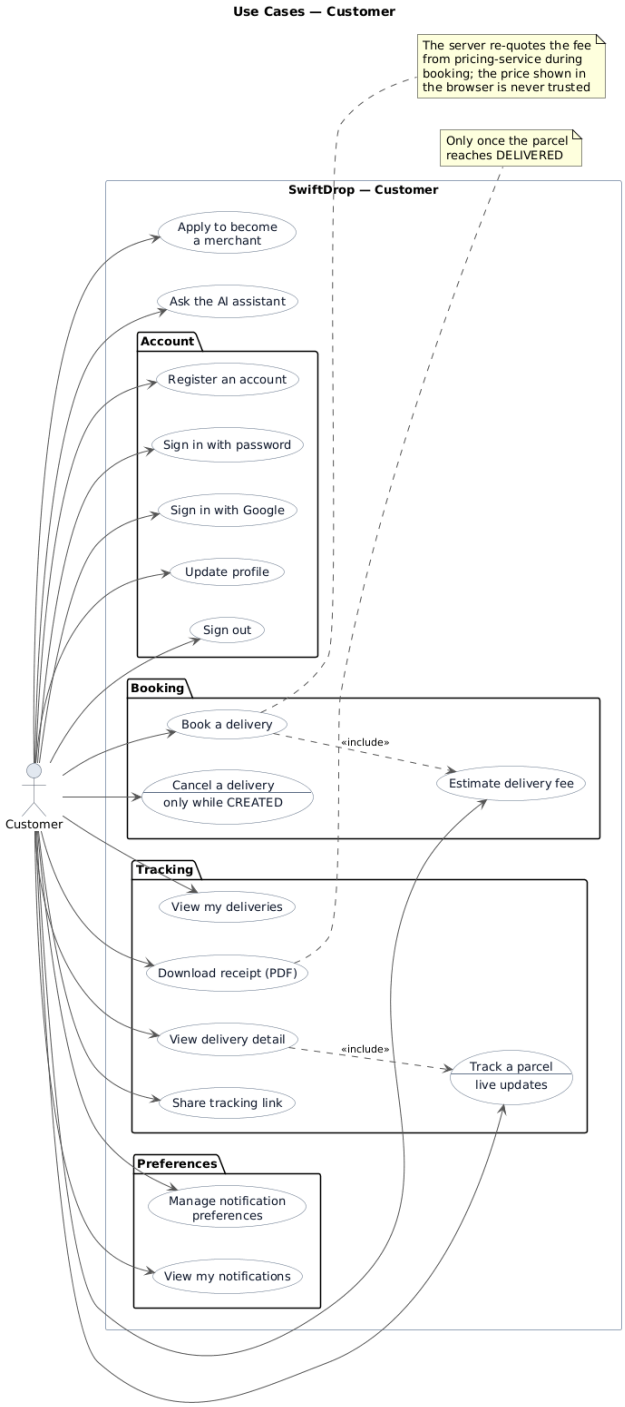


Figure 2.2: *Customer use cases. Booking* includes *fee estimation because the server re-quotes the price during booking rather than trusting the browser*

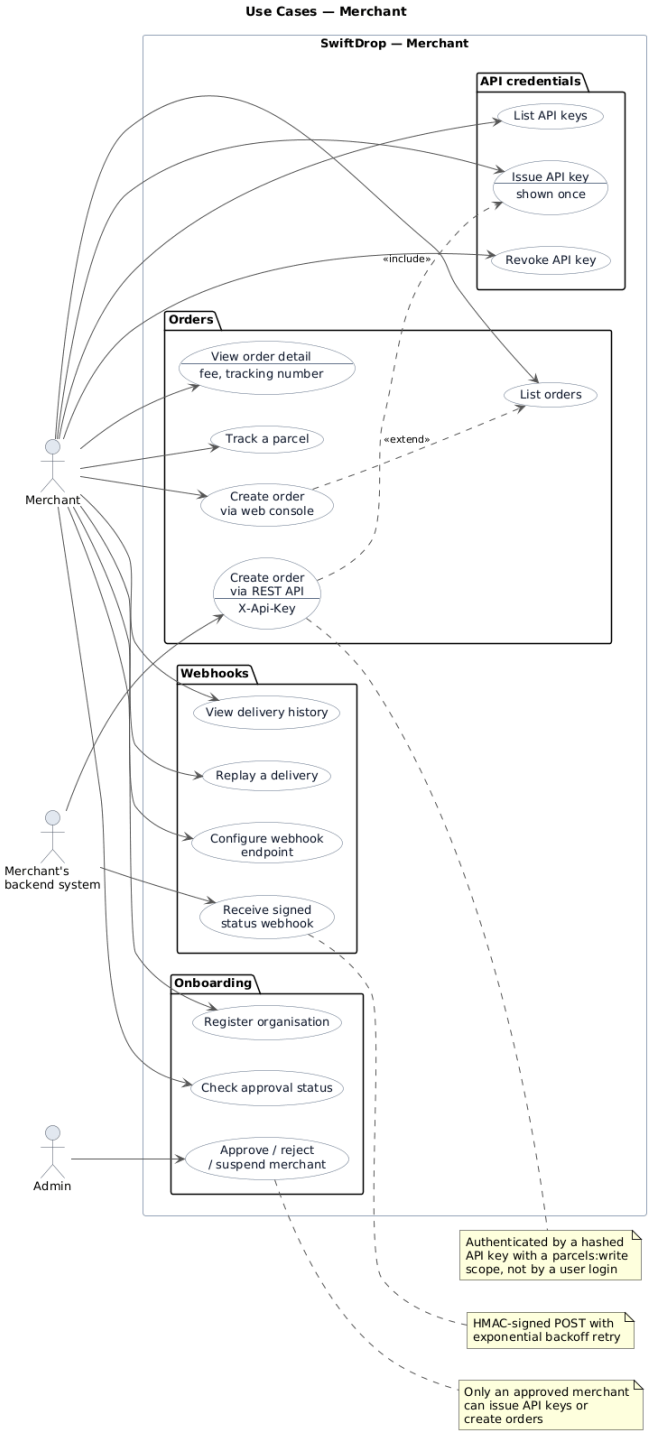


Figure 2.3: *Merchant use cases, distinguishing the human console path from the server-to-server REST API path authenticated by a scoped API key*

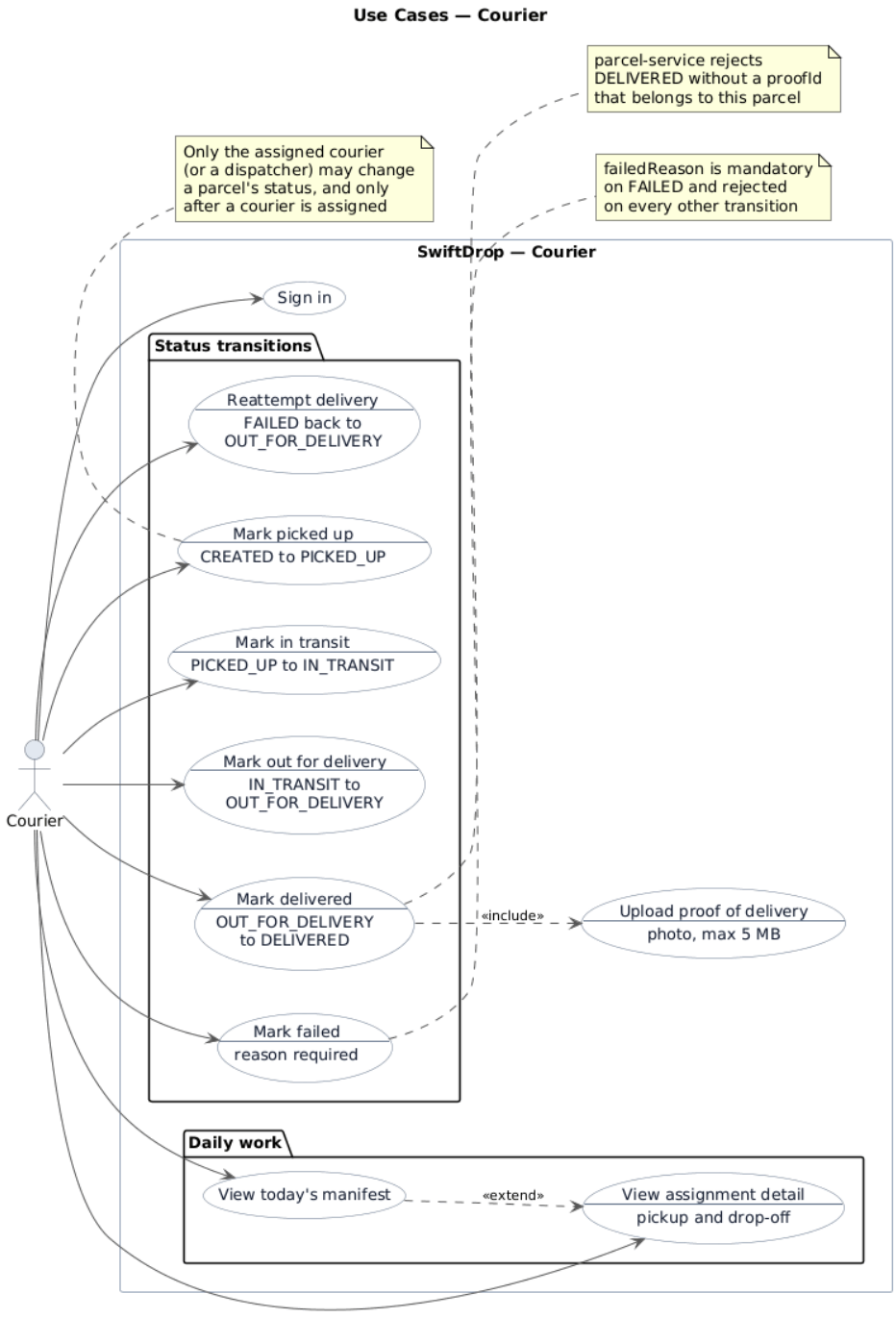


Figure 2.4: *Courier use cases. Every transition is guarded; marking a parcel delivered* includes *uploading proof of delivery*

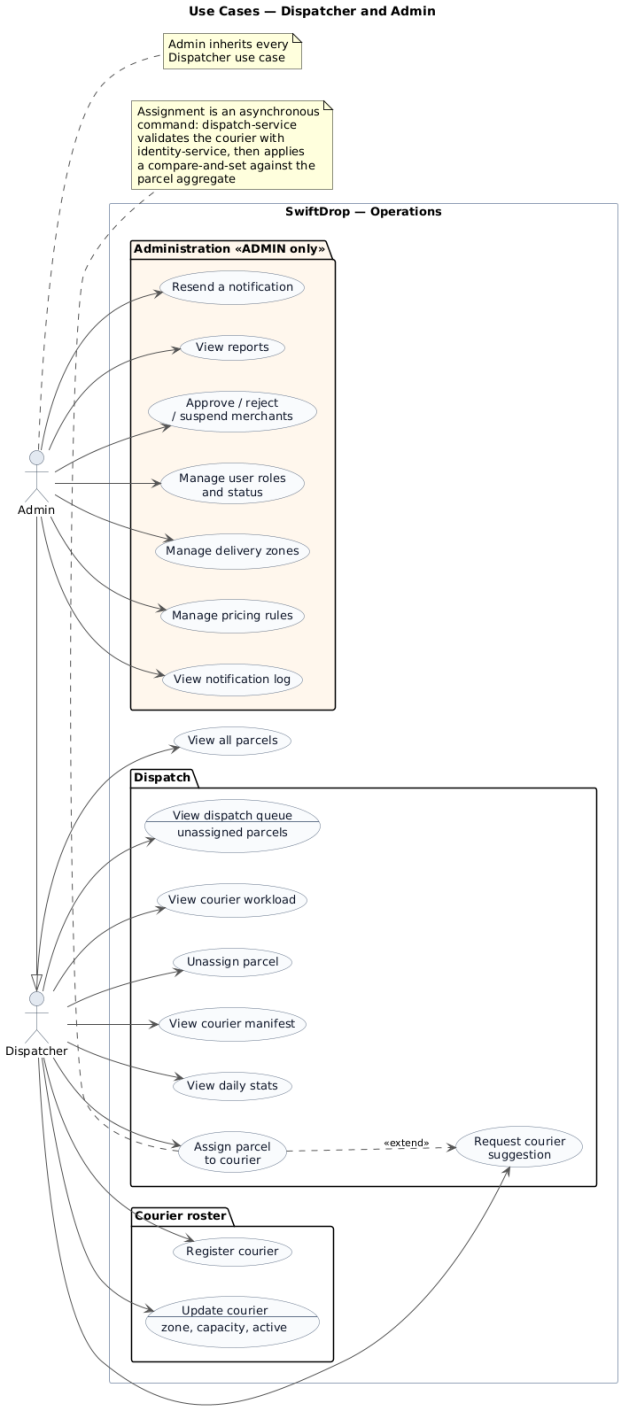


Figure 2.5: *Dispatcher and administrator use cases, with the administrator-only region shaded*

Chapter 3

System Architecture

3.1 Architectural Style and Justification

SwiftDrop is built as **microservices behind an API gateway that also serves as a backend-for-frontend**, with a strict three-layer structure inside every service.

The boundaries were derived from the domain, not from the technical layering: each service owns a capability that changes for its own reasons. Pricing rules change when commercial terms change; the parcel lifecycle changes when operations change; notification channels change when a provider changes. There is no "controller service" or "database service".

1. **Team independence.** Ten engineers developed and released their domains without code contention.
2. **Blast-radius containment.** A spike in public tracking traffic or a failure in the notification pipeline does not affect booking or authentication.
3. **Storage fit per context.** Tracking needs a fast, expiring, read-optimised projection and uses Redis with a time-to-live, which would be awkward to justify inside a shared relational schema.
4. **Independent scaling.** Read-heavy tracking and write-heavy booking scale on different curves.

3.1.1 Honest Cost of This Choice

A modular monolith with the same internal boundaries would have been a defensible - arguably better - choice at this stage, and the costs microservices imposed are visible in the codebase.

Cost	Where it appears
No cross-service transactions	Creating a merchant order writes to two databases through an HTTP call; consistency is recovered with a transactional outbox, deterministic idempotency keys and compensating reads
User-visible eventual consistency	The dispatch board and tracking page are event-fed projections; they are correct eventually and the interface must tolerate the gap
Duplicated cross-cutting config	Every service repeats JWT decoder and resource-server setup; a change to token validation is an eight-service change
Contracts not compiler-checked	One service writes a JWT claim as `merchantId` while three others read `merchant_id` (Section 10.2); in a single codebase this is a compile error
Operational surface	Six databases, a broker, Redis, an object store and a mail server must all run before the system works end to end

Table 3.1: *Costs incurred by the microservices decision*

The mitigation applied is a shared contracts module holding event schemas, shared enumerations and error codes, with golden-file tests pinning the JSON shapes. It narrows the gap without closing it.

3.2 Layered Topology

The platform is organised into four layers.

Presentation
A React single-page application. It holds no business rules - route guards mirror server roles for user experience only, and every decision is re-made server-side. The access token is held in memory, never in browser storage.

Edge Spring Cloud Gateway on WebFlux: the only publicly reachable component and the single place implementing JWT validation (issuer and audience), CORS, security headers, Redis token-bucket rate limiting, Resilience4j circuit breakers, request-id propagation and identity header injection. Internal routes are denied outright at the edge.

Business logic
Eight services, each owning a bounded context, each internally layered as `api` → `service` → `repository` → `domain`. Dependencies point inward only; controllers carry no business rules and entities never leave the service layer.

Data Six PostgreSQL databases, Redis, RabbitMQ and MinIO.

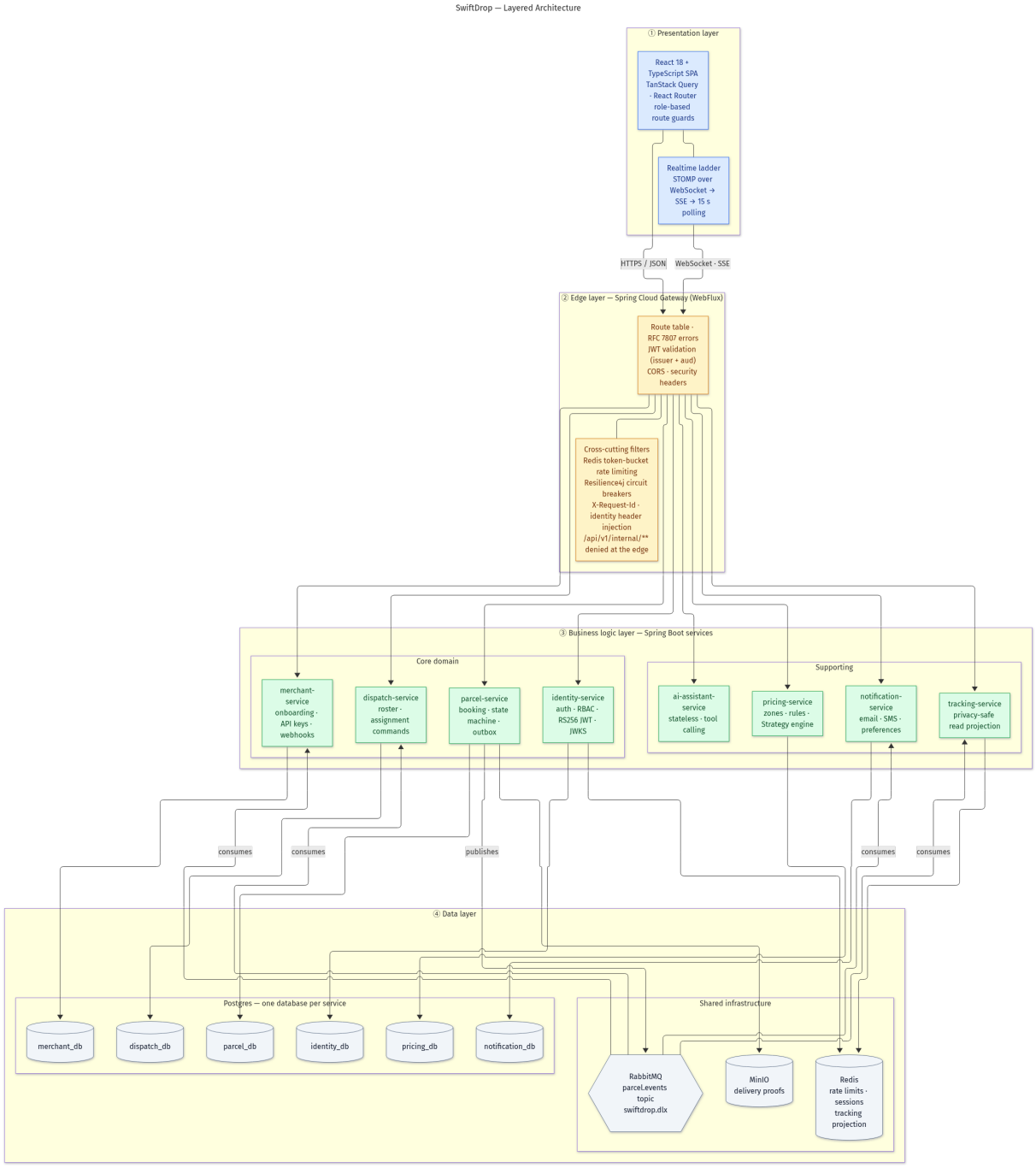


Figure 3.1: *Layered architecture showing the presentation, edge, business-logic and data layers, with synchronous calls solid and event flow dashed*

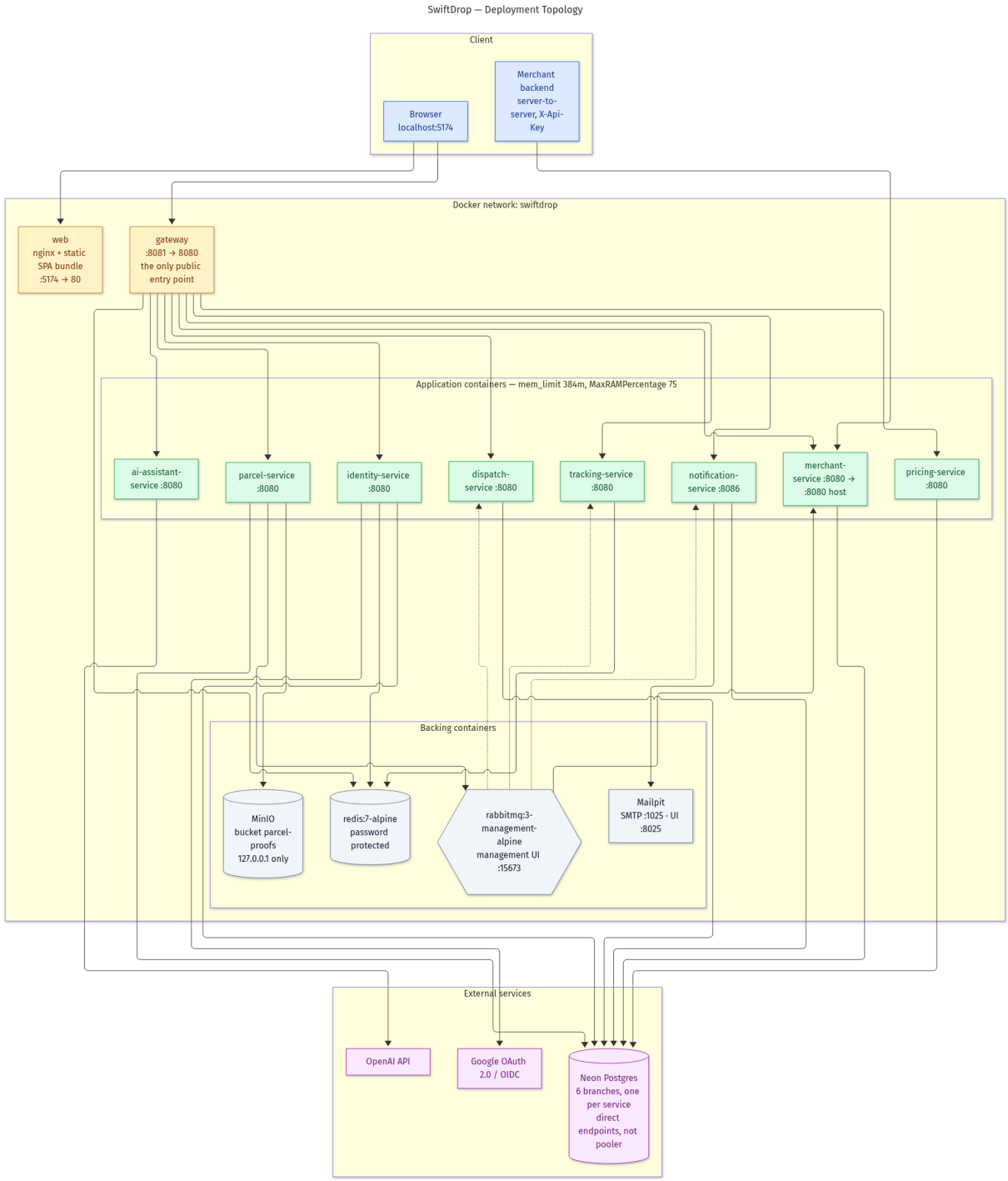


Figure 3.2: *Deployment topology: container boundaries, host port mappings and external dependencies*

3.3 Service Inventory

Service	Responsibility
gateway	Edge routing, JWT validation, CORS, rate limiting, circuit breaking, identity header injection, aggregated API documentation
identity-service	Registration, login, Google OIDC, RS256 token issuance, JWKS publication, refresh rotation, role administration

Service	Responsibility
parcel-service	Parcel aggregate, booking, guarded state machine, proof upload, PDF receipts, transactional outbox
pricing-service	Zones, city resolution, temporal pricing rules, Strategy-based fee engine
dispatch-service	Courier roster, dispatch queue projection, assignment commands, workload accounting
merchant-service	Merchant onboarding and approval, API key lifecycle, order ingestion, signed outbound webhooks with retry
tracking-service	Privacy-reduced read projection in Redis; STOMP and SSE push
notification-service	Event-driven email and SMS dispatch, per-user preferences, delivery log, retry scheduler
ai-assistant-service	Stateless conversational assistant exposing read-oriented domain tools over SSE

Table 3.2: *Service inventory and responsibilities*

Chapter 4

Database and Entity-Relationship Modelling

4.1 Database-per-Service

SwiftDrop implements database-per-service strictly. Six services own a PostgreSQL database each; tracking-service holds its projection entirely in Redis, and ai-assistant-service has no persistence at all.

The consequence that shapes the whole model is that **no service holds a foreign key into another service's database**. Relationships crossing a service boundary exist only as UUID columns enforced in application code. The identity `users.id` column is the most-referenced value in the system - thirteen columns across four other databases point at it - and it has no inbound foreign keys whatsoever.

> **Diagram convention.** A solid line is a real foreign key enforced by PostgreSQL. A dashed line is a cross-database logical reference enforced only by application code.

4.2 The 24-Table Domain Model

Service	Database	Tables
identity-service	`identity_db`	`users`, `roles`, `user_roles`, `refresh_tokens`
parcel-service	`parcel_db`	`parcels`, `parcel_status_history`, `delivery_proofs`, `parcel_outbox`, `parcel_assignment_commands`
merchant-service	`merchant_db`	`merchants`, `api_keys`, `oauth_clients`, `merchant_orders`, `webhook_deliveries`
dispatch-service	`dispatch_db`	`couriers`, `dispatch_parcels`, `assignments`, `assignment_commands`, `processed_events`
pricing-service	`pricing_db`	`zones`, `zone_cities`, `pricing_rules`
notification-service	`notification_db`	`notifications`, `notification_preferences`
tracking-service	Redis only	`tracking:parcel:*`, `tracking:timeline:*` (90-day time-to-live)
ai-assistant-service	None	In-memory conversations, 24-hour expiry

Table 4.1: *Table ownership across the six service databases*

Points worth drawing out:

- **Role-based access control** uses the classic three-table design. Role identifiers are the enumeration names themselves, so a token's `roles` claim maps directly onto rows.

- **`parcels` snapshots its fee.** Re-reading a parcel years later yields the price actually charged, not what current rules would produce.
- **`parcel_status_history` is an append-only audit trail** recording the from-status, to-status, acting user, and an optional proof or failure reason for every transition.
- **`couriers.id` is not `users.id`.** A courier has a dispatch-local surrogate key, and `parcels.assignee_id` points at the *user*, so joining a parcel to a courier always passes through the user identifier.
- **Pricing is temporal.** Each rule carries an effective period, and a database trigger forbids overlapping active periods for the same route and size band.

4.3 Entity-Relationship Diagrams

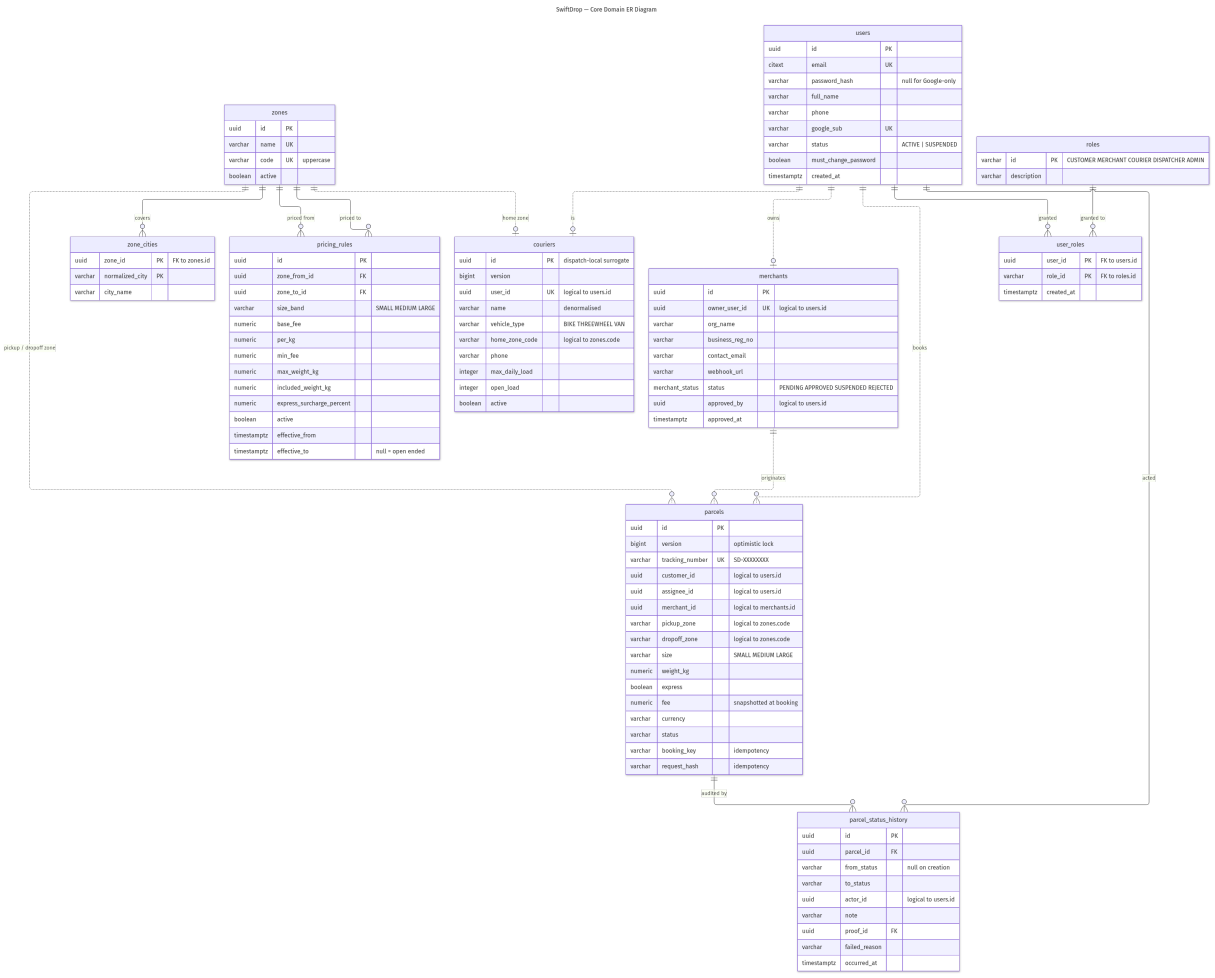


Figure 4.1: *Core domain model covering users, roles, merchants, parcels, status history, couriers, zones and pricing*

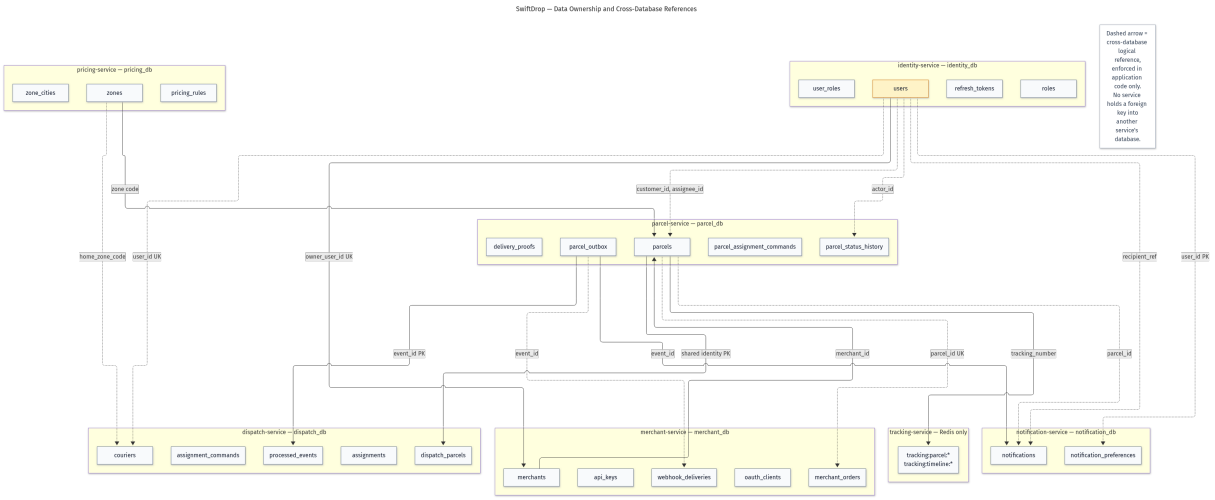


Figure 4.2: *Data ownership map: which service owns which tables, and every logical reference that crosses a database boundary*

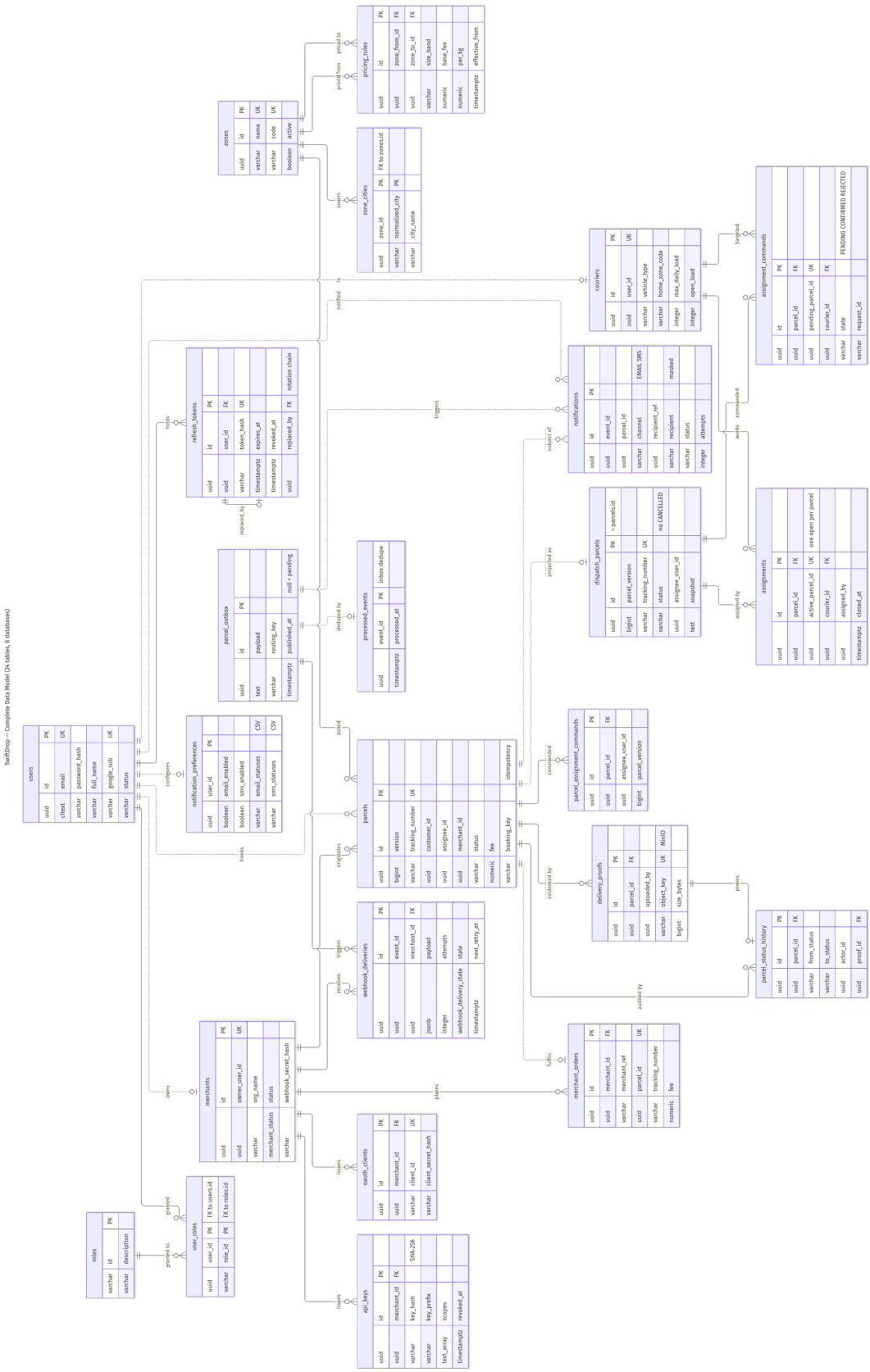


Figure 4.3: *Complete 24-table data model, including the infrastructure tables that make the asynchronous integration safe: the transactional outbox, the inbox deduplication table, the webhook delivery log and the assignment command tables*

4.3.1 Known Model Drift

Two asymmetries are visible in the migrations and are recorded here rather than smoothed over:

1. `dispatch_parcels.status` carries a check constraint rejecting `CANCELLED`, while `parcels.status` accepts it, so a cancelled parcel cannot be projected into the dispatch read model.
2. `parcels.pickup_zone` is `VARCHAR(80)` while its logical source `zones.code` is `VARCHAR(12)` - the consumer's column is wider than the producer's.

Chapter 5

Integration Architecture and Messaging

5.1 Integration Patterns

Pattern	Where used	Why this pattern
Request/reply	parcel → pricing; dispatch → identity and parcel; gateway → all	The caller cannot proceed without the answer: a parcel cannot be persisted before an authoritative fee exists
Publish/subscribe	`parcel.events` topic exchange: one producer, four consumers	A status change has several independent audiences; the producer should not know who they are
Message broker	RabbitMQ topic exchange plus `swiftdrop.dlx`	Durable, routable fan-out with retry and dead-lettering
Transactional outbox	`parcel_outbox` written inside the business transaction, drained by a scheduled publisher	Removes the dual-write problem: state change and publish intent commit atomically
Idempotent consumer	`processed_events`; a Redis key with TTL; a unique constraint on event, channel and recipient	Publisher confirms give at-least-once delivery, so every consumer must tolerate replay
Idempotency key	`Idempotency-Key` header on booking; deterministic key for merchant orders	A retried booking returns the original parcel rather than creating a duplicate
CQRS read model	`dispatch_parcels`; the Redis tracking projection	Fast reads in a different shape from the write model, without querying the owning service
API gateway / BFF	Spring Cloud Gateway and the dashboard fan-out	One edge for authentication, CORS, rate limiting and circuit breaking
Circuit breaker	Resilience4j on every gateway route	Prevents a sick upstream from cascading
Client credentials	`POST /api/v1/auth/service-token`	Internal calls authenticate as a service, carrying no human roles
Token forwarding	parcel → pricing; assistant → parcel and pricing	Preserves the user's own authorisation instead of escalating to a service identity
Outbound webhook	merchant-service to the merchant endpoint, HMAC-signed	Push integration for partners who cannot poll
Fallback ladder	STOMP over WebSocket → SSE → 15-second polling	Live updates must survive proxies that break WebSockets

Table 5.1: *Integration patterns and the rationale for each*

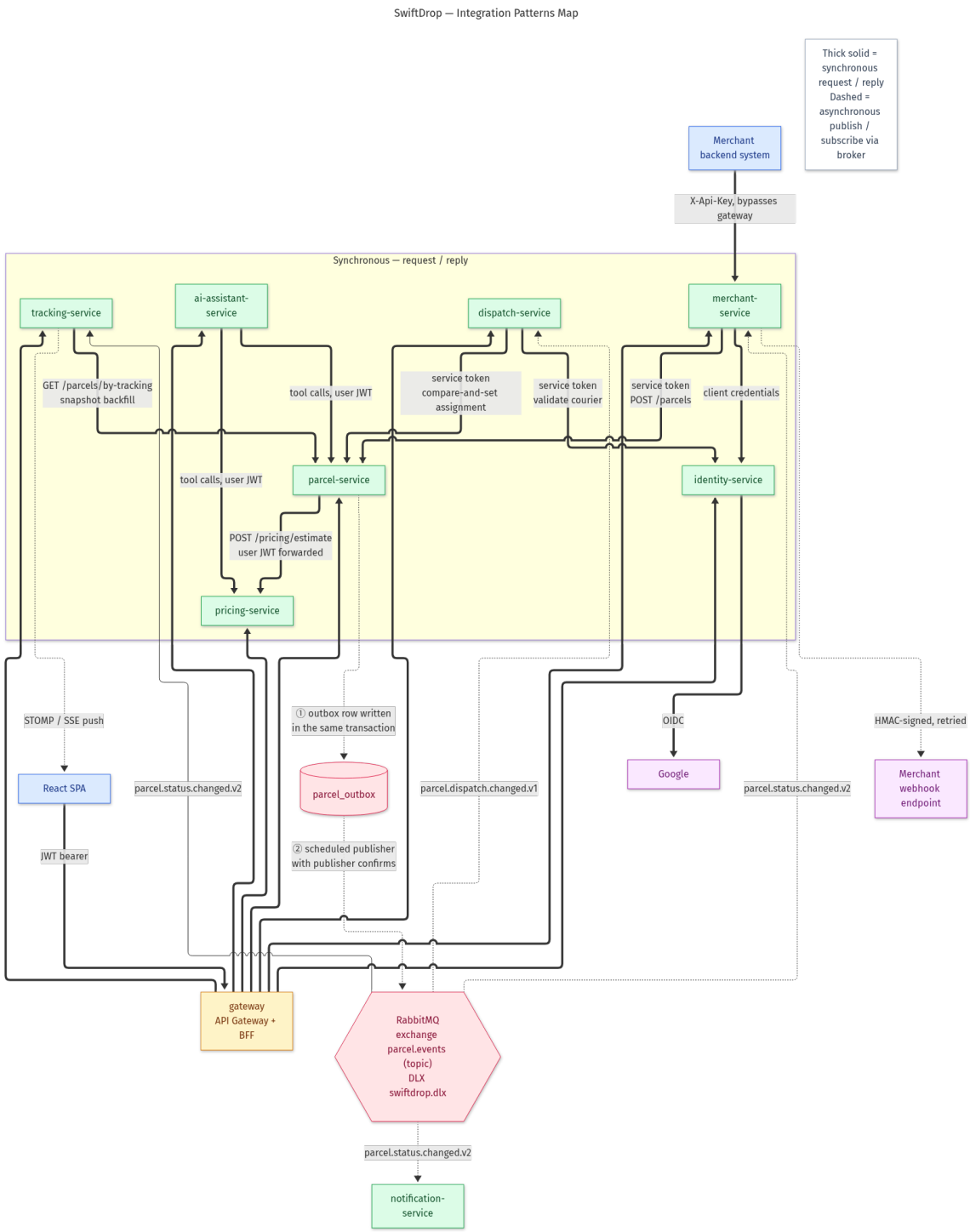


Figure 5.1: *Integration patterns map. Thick solid edges are synchronous request/reply; dashed edges are asynchronous publish/subscribe through the broker*

5.2 Messaging Topology and Delivery Guarantees

Every message is wrapped in a common envelope, `EventEnvelope<T>`, carrying `eventId`, `eventType`, `version`, `occurredAt`, `producer` and the payload. The `eventId` is the deduplication key every consumer uses.

Naming is mechanical: routing keys are `<eventType>.v<version>` and queues are `<consuming-service>.<routing-key>`, with `.dlq` appended for dead letters. Reading a

queue name reveals both its consumer and its payload.

Routing key	Queue	Consumer
`parcel.dispatch.changed.v1`	`dispatch.parcel.dispatch.changed.v1`	dispatch-service
`parcel.status.changed.v2`	`tracking.parcel.status.changed.v2`	tracking-service
`parcel.status.changed.v2`	`notification.parcel.status.changed.v2`	notification-service
`parcel.status.changed.v2`	`merchant-webhook.parcel.status.changed.v2`	merchant-service

Table 5.2: *Bound queues on the `parcel.events` topic exchange*

A version 1 status event (`parcel.status.changed.v1`) is declared in the shared contracts module but is *never bound or consumed*. Version 2 is the event actually produced. Note also that dispatch-service consumes a different event entirely - `parcel.dispatch.changed.v1` - rather than the status event.

The guarantee is **at-least-once**. The producer writes the outbox row inside the business transaction; the publisher sends persistently and waits for a broker confirmation before marking the row published; on a negative acknowledgement or return it stops the batch and leaves the row for the next cycle. Duplicates are therefore possible by design, which is why every consumer deduplicates.

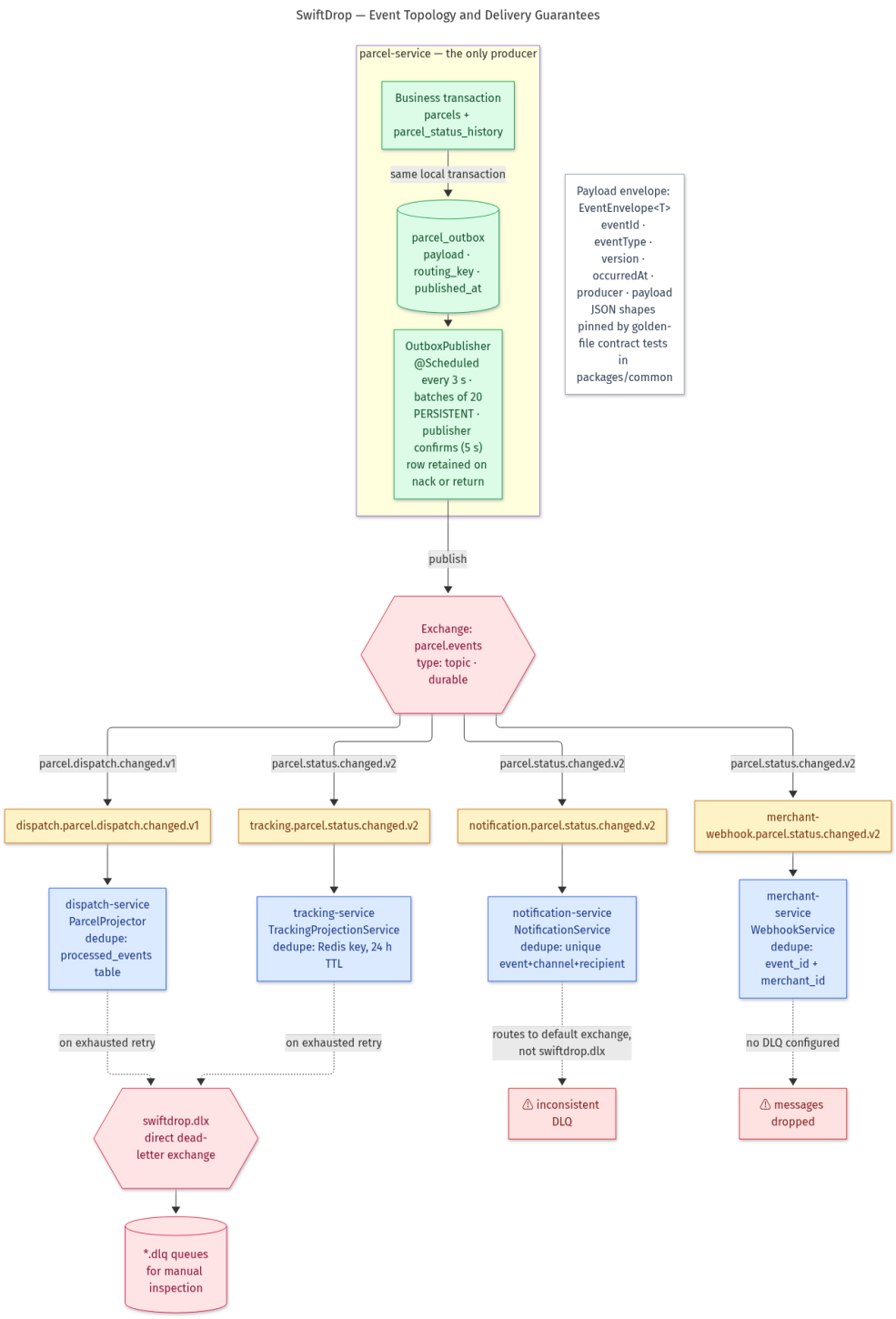


Figure 5.2: *Event topology from the transactional outbox through the topic exchange to the four bound queues and the dead-letter exchange*

Chapter 6

Core Workflows and State Transitions

6.1 Customer Booking and Dynamic Pricing

Two aspects deserve attention. First, the fee is quoted twice - once for display while the customer types, and again authoritatively inside the booking transaction, so the browser's number is never trusted. Second, four writes and two outbox rows commit in a single local transaction, so an event can never be published for a parcel that failed to save, and a saved parcel can never silently fail to notify.

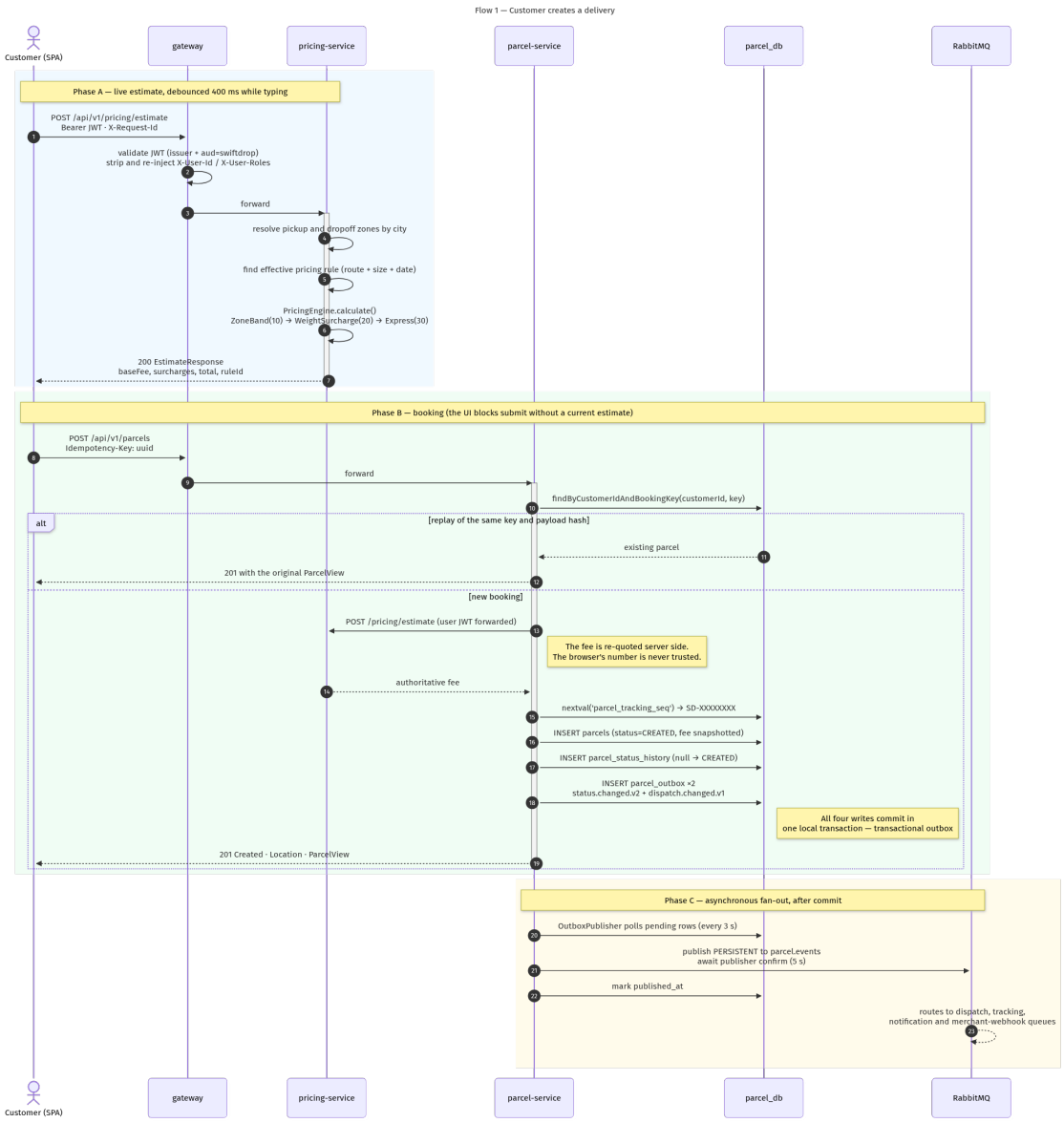


Figure 6.1: *Customer booking: debounced estimate, server-side re-quote, atomic persistence with the transactional outbox, and asynchronous fan-out*

6.2 Merchant Order Creation via the REST API

The server-to-server path. The API key is never stored - only its SHA-256 hash and an eight-character prefix. Verification looks candidates up by prefix, filters by expiry and scope, and compares hashes in constant time to avoid leaking information through response timing.

Because no user is present, merchant-service obtains a short-lived service token from identity-service before calling parcel-service. The idempotency key it sends is *deterministic*, derived from the merchant identifier and the merchant's own reference, so a retry from the partner's system reuses the same parcel rather than creating a second one.

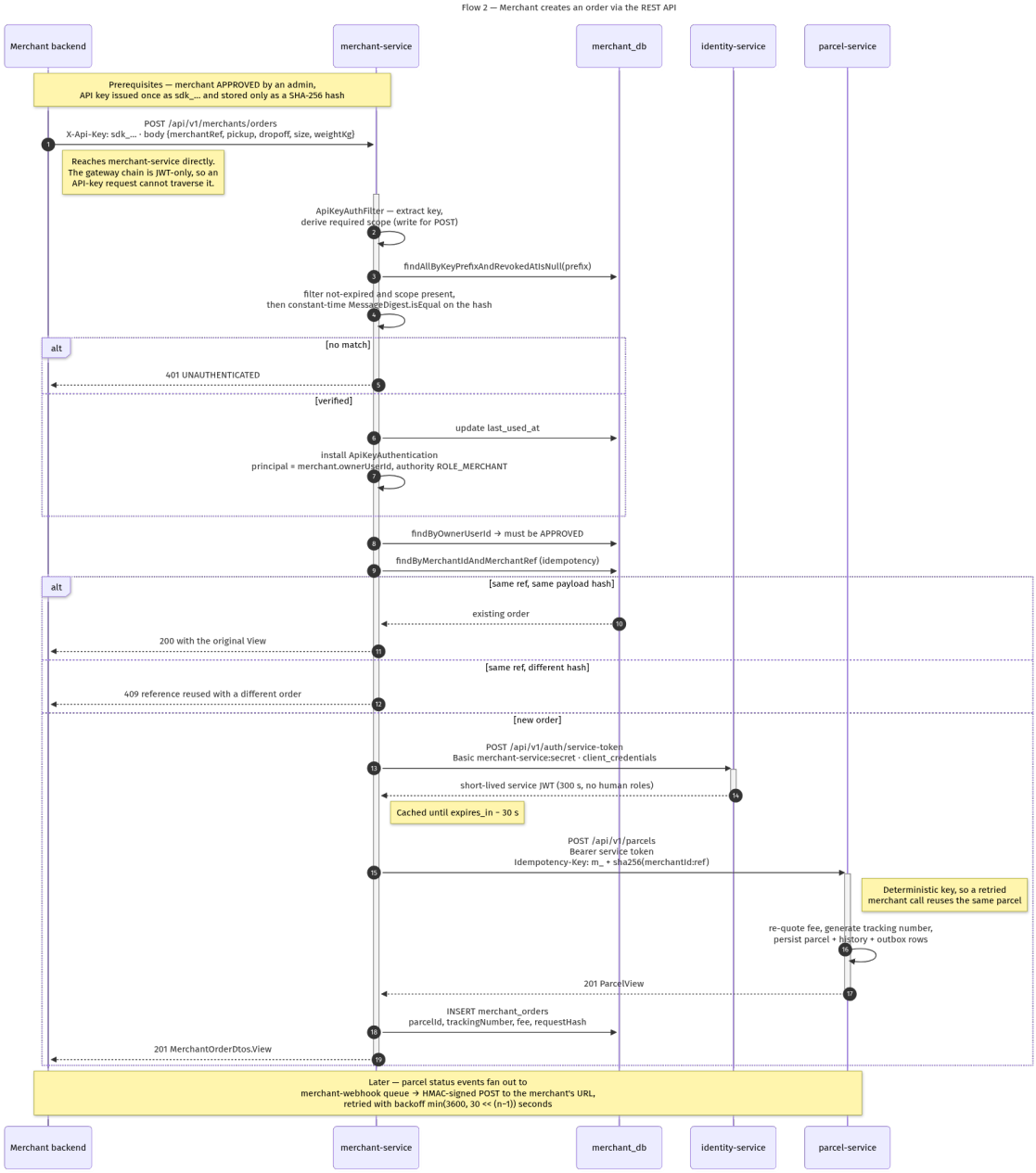


Figure 6.2: *Merchant REST API order ingestion with API key verification, service-token acquisition and deterministic idempotency*

6.3 Courier Status Update Triggering Notifications

The clearest illustration of the publish/subscribe design: one status update fans out to three unrelated consumers, none of which the courier's request waits for.

Two deliberate orderings are visible. Proof of delivery is uploaded *before* the status change, so a failed upload leaves the parcel untouched and the courier simply retries. The notification recipient is taken from the event's `customerUserId` - the parcel's owner - not from the courier who triggered the change.

> Proof-of-delivery objects are uploaded to MinIO by **parcel-service**, which owns the `ProofStorage` abstraction. Dispatch-service is not involved in the upload path.

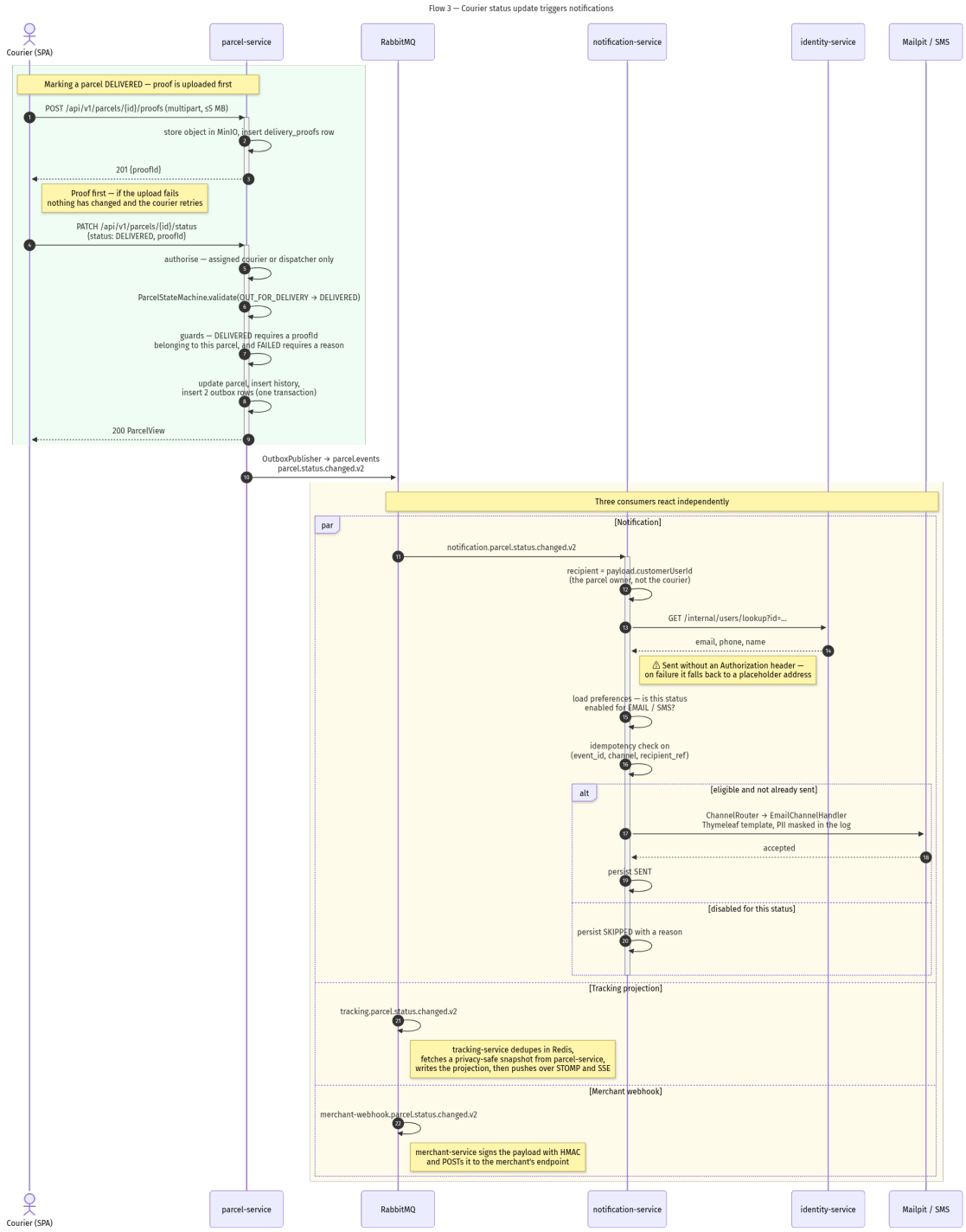


Figure 6.3: *Courier status update fanning out to the notification pipeline, the tracking projection and the merchant webhook*

6.4 Google OAuth 2.0 Login with PKCE

Authorization Code flow with PKCE. The browser is a public client and cannot hold a secret, so PKCE binds the authorisation code to the client that requested it: the application generates a random verifier, transmits only its SHA-256 hash, and must present the original verifier to redeem the code.

The defences worth naming are that `state` is single-use and consumed atomically from Redis;

the verifier is compared in constant time; the redirect URI is re-checked against the one recorded at authorisation time; and the identity token's issuer and audience are verified before any account is touched. An account provisioned through Google receives a null password hash, so password login against it can never succeed.

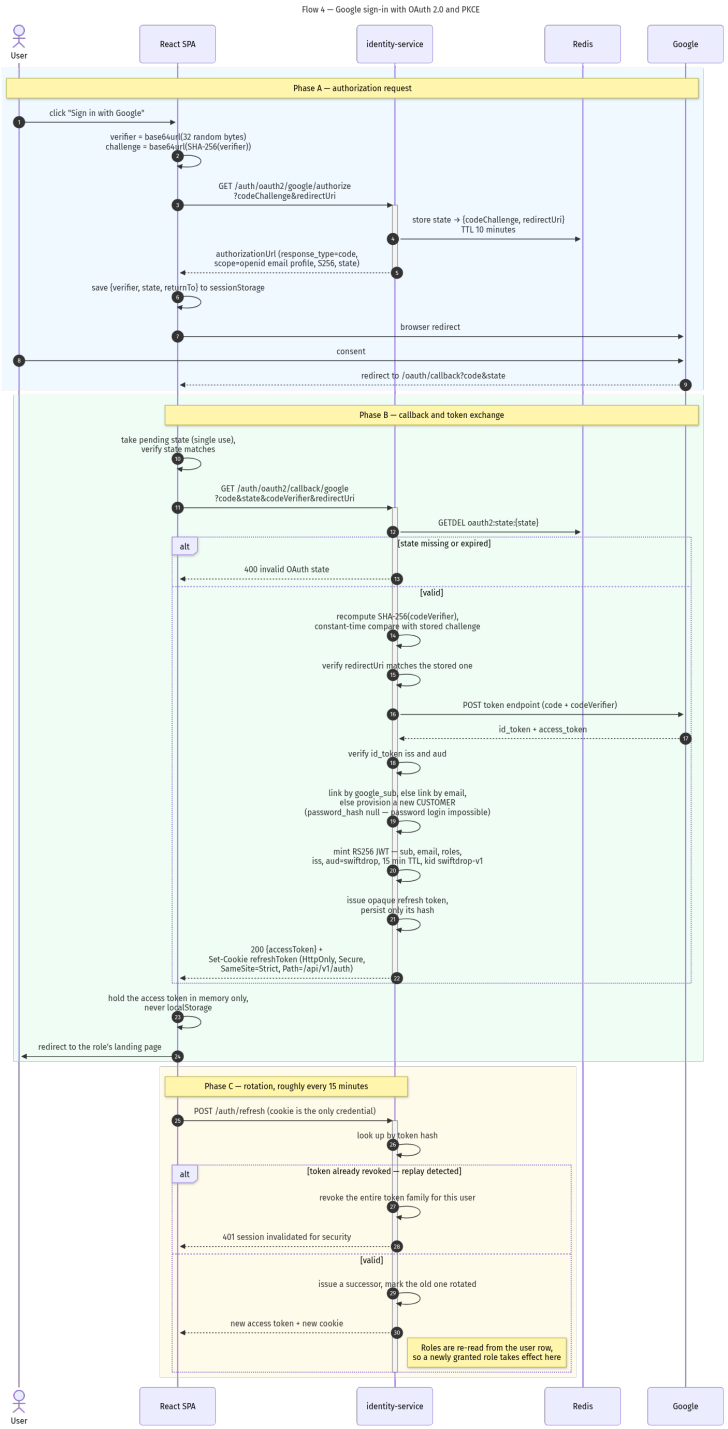


Figure 6.4: *Google OAuth 2.0 PKCE login, including token exchange, account link-or-provision, and refresh-token rotation with replay detection*

6.5 Parcel Lifecycle State Machine

Transitions are validated against a single declarative table in `ParcelStateMachine` rather than scattered conditionals.

From	Permitted to	Guard
`CREATED`	`PICKED_UP`, `CANCELLED`	Cancellation is legal only here
`PICKED_UP`	`IN_TRANSIT`	-
`IN_TRANSIT`	`OUT_FOR_DELIVERY`	-
`OUT_FOR_DELIVERY`	`DELIVERED`, `FAILED`	`DELIVERED` requires a `proofId` belonging to this parcel; `FAILED` requires a reason
`FAILED`	`OUT_FOR_DELIVERY`	Reattempt
`DELIVERED`	-	Terminal; enables receipt download
`CANCELLED`	-	Terminal

Table 6.1: *Permitted parcel state transitions*

`FAILED` is not a terminal state. A failed delivery returns to `OUT_FOR_DELIVERY` for a reattempt, so the transition graph contains a cycle and the progression is not strictly linear. Only `DELIVERED` and `CANCELLED` are terminal.

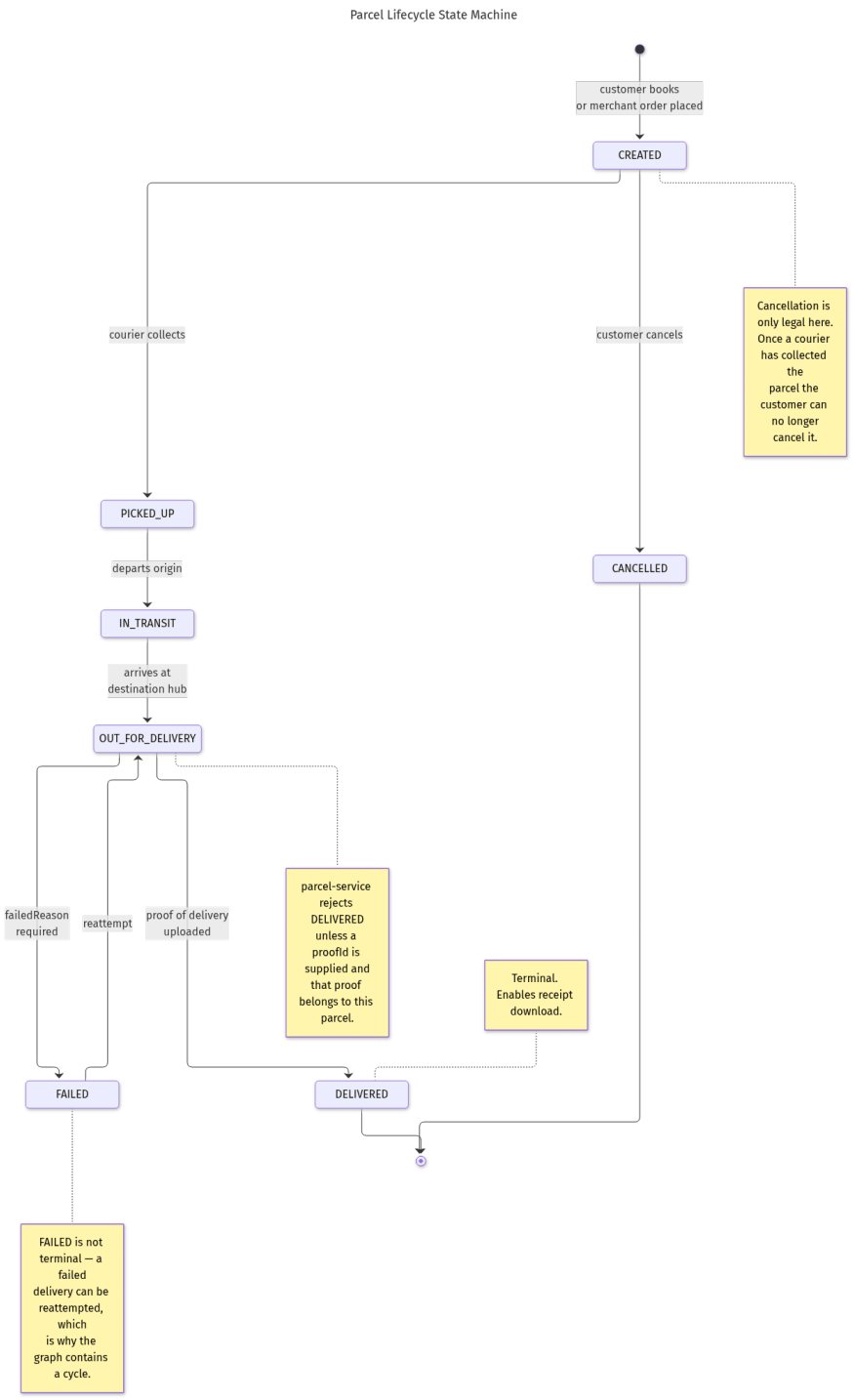


Figure 6.5: *Parcel lifecycle state machine transcribed from the implementation, including the failure-reattempt cycle*

Chapter 7

AI Assistant Service

7.1 Architecture

SwiftDrop includes an AI assistant service built on Spring AI. A large language model is given a set of callable domain tools; the model decides which to invoke in response to a natural-language prompt, and the service streams both its reasoning and each tool invocation back to the browser over Server-Sent Events.

The service is stateless with respect to durable storage: conversations are held in memory with a 24-hour expiry and are scoped to the calling user.

7.2 Implemented Tools

Tool	Status	Behaviour
`estimateFee`	**Implemented**	Calls pricing-service to quote a fee from origin, destination, size band, weight and express flag
`trackParcel`	**Implemented**	Retrieves the tracking timeline for a tracking number, returning a privacy-reduced view without contact or street-address detail
`listMyParcels`	**Implemented**	Lists the authenticated customer's own parcels; injected only when the caller holds the customer role

Table 7.1: *Tools currently exposed to the model*

All three are read-oriented. The assistant can answer questions about the system but cannot presently modify state.

7.3 Designed but Not Yet Implemented

The following capabilities were designed as part of the agentic vision and are retained here as roadmap items. None is currently callable.

Capability	Status	Intended behaviour
`createParcel`	**Planned**	Parse a conversational booking instruction and persist a parcel end to end, returning a tracking number
`assignParcel`, `inspectWorkload`	**Planned**	Evaluate the unassigned queue and courier capacity, then propose or trigger assignments
`generateApiKey`, `retryWebhook`	**Planned**	Allow merchants to provision keys and re-drive failed webhook deliveries conversationally
MCP server exposure	**Partial**	The MCP starter dependency is declared, but no server wiring exists; the tools are not yet reachable over the Model Context Protocol
General per-authority tool filtering	**Partial**	A single-role check currently gates `listMyParcels`; a caller holding several roles loses that tool

Table 7.2: *Planned assistant capabilities retained in the roadmap*

7.4 Security of the Tool Path

When the model invokes a tool, the service propagates the caller's original bearer token to the downstream microservice. Existing Spring Security authorisation therefore applies unchanged - the assistant cannot read anything the user could not read directly through the API. Tool invocations are streamed to the client before execution, giving the user visibility of exactly what the assistant is doing on their behalf.

Chapter 8

Design Patterns and Engineering Principles

8.1 Strategy Pattern in the Pricing Engine

The clearest pattern application in the codebase. `PricingEngine` depends only on the `PricingStrategy` interface and receives every implementation as an injected list, sorted by each strategy's declared order. Calculation reduces over that list, threading a breakdown through each strategy in turn.

Strategy	Order	Contribution
`ZoneBandStrategy`	10	Sets the base fee from the matched pricing rule
`WeightSurchargeStrategy`	20	Charges only weight above the rule's included allowance
`ExpressSurchargeStrategy`	30	Adds the configured express percentage

Table 8.1: *Ordered pricing strategy chain*

Introducing a fuel surcharge means adding one Spring component and nothing else: the engine does not change, and neither does any existing strategy. The same shape appears again in notification-service, where a channel router selects from an injected list of channel handlers.

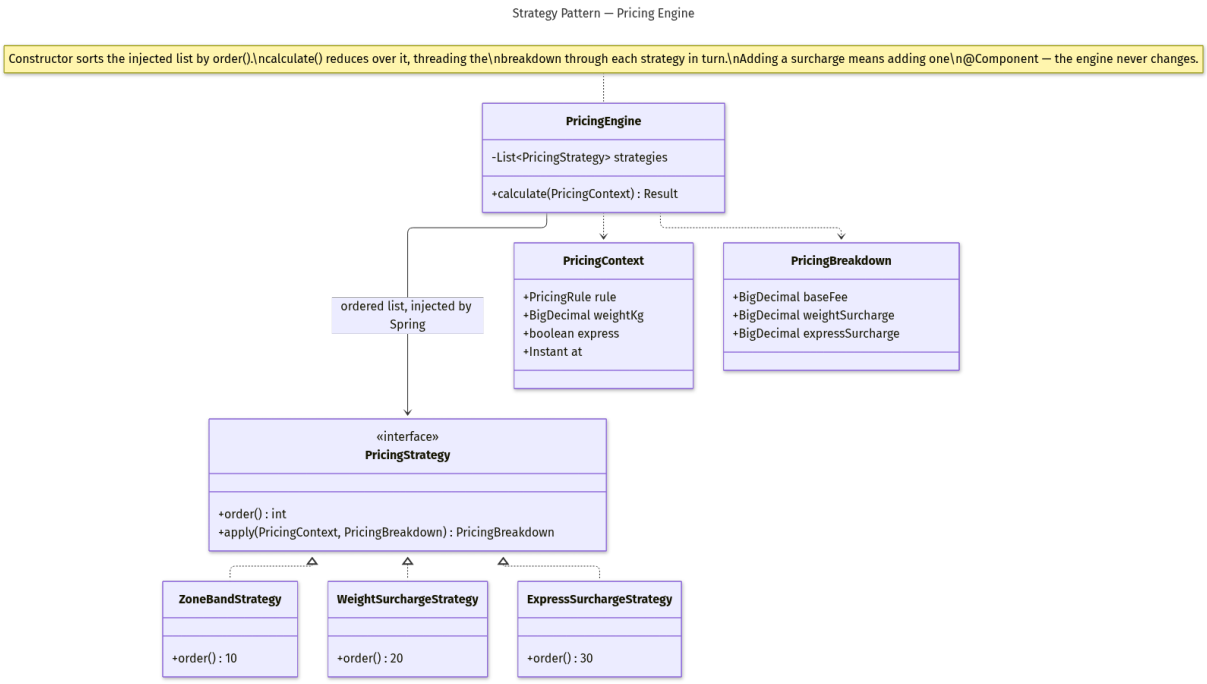


Figure 8.1: *Strategy pattern in the pricing engine*

8.2 SOLID Principles

Principle	Evidence in the codebase
Single responsibility	One bounded context per service; within each, controllers handle HTTP, services hold rules, repositories handle persistence
Open/closed	The pricing engine and the notification channel router accept new behaviour without modification
Liskov substitution	Every `PricingStrategy` is interchangeable; the engine makes no assumption about which concrete strategy it holds
Interface segregation	`PricingStrategy` declares two methods; `ProofStorage` exposes only what parcel-service needs, which is what makes the MinIO backend swappable
Dependency inversion	High-level policy depends on abstractions the framework satisfies: the engine on the strategy interface, services on Spring Data interfaces, and parcel-service on a pricing client interface rather than on an HTTP client

Table 8.2: *SOLID principles as applied*

8.3 Further Patterns Applied

Pattern	Implementation	Purpose
State machine	`ParcelStateMachine`	Legal transitions declared once in a table a reader can verify at a glance
Repository	Spring Data JPA interfaces	Business code expresses intent rather than SQL
Transactional outbox	`OutboxEvent` and a scheduled publisher	Atomic state change and event intent
Observer / publish-subscribe	AMQP listeners	Services react to parcel events without the producer knowing them
Chain of responsibility	Spring Security filter chains; the API key filter ahead of the JWT filter	A request passes through independent, individually testable concerns
Projection	The dispatch projector and the Redis tracking repository	Read models shaped for their consumer and reduced for privacy
Data transfer object	View records at every boundary	Entities never serialise directly, so the wire contract is deliberate and PII can be excluded
Value object	Tracking number; the embeddable address	Validity enforced at construction
Optimistic locking	Version columns on parcel, courier, assignment and pricing rule	Concurrent edits conflict loudly instead of silently overwriting

Table 8.3: *Design patterns applied across the platform*

8.4 Don't Repeat Yourself

A shared contracts module is the single place where cross-service contracts live: event payload records, the event envelope, exchange and queue constants, the shared role, status and size enumerations, and the error-code catalogue behind RFC 7807 responses. Queue names are *derived* from routing keys rather than written twice, and golden-file tests pin each event's JSON shape so a field rename that would break a consumer fails the producer's build.

The limit is worth stating: this holds for anything expressible as shared Java, but not for configuration. The JWT decoder setup is repeated in all eight services, and the claim-name defect in Section 10.2 is exactly the kind of error that duplication permits.

Chapter 9

Security Architecture

9.1 Cryptographic Authentication

Control	Implementation
Token signing	RSA 2048-bit, RS256, key identifier `swiftdrop-v1`
Key discovery	RFC 7517 JWKS published at `/.well-known/jwks.json`, cached 24 hours; resource servers validate locally without contacting the identity database
Access token lifetime	15 minutes
Refresh token	7 days, opaque, stored only as a SHA-256 digest, rotated on every use
Replay detection	Presenting an already-rotated token revokes the entire token family for that user
Cookie transport	`HttpOnly`, `Secure`, `SameSite=Strict`, path-scoped to the authentication endpoints

Table 9.1: *Authentication controls*

9.2 Defence in Depth

Control	Implementation
Password hashing	BCrypt at work factor 10 with a secure random salt
Credential enumeration	A missing email still executes a constant-time dummy verification, and invalid email and invalid password return an identical problem document
Rate limiting	Redis token buckets at the edge: 5 requests per minute on login, 30 per minute on public tracking, 100 per minute on other API routes. Keyed by user when authenticated, by address otherwise, and fails open if Redis is unreachable
API key storage	SHA-256 hash and an eight-character prefix only; verification compares in constant time
Proof-of-delivery access	MinIO presigned URLs valid for 300 seconds
PII protection	Recipient addresses masked in the notification log; a log converter redacts emails and phone numbers across all logger statements; the tracking projection excludes addresses and phone numbers by construction
Identity header integrity	The gateway strips any client-supplied identity header and rederives it from a validated token
Transport hardening	CORS allowlist, `X-Frame-Options`, `nosniff`, HSTS and a content security policy at the edge

Table 9.2: *Defence-in-depth controls*

Chapter 10

Quality Assurance and Defect Transparency

10.1 Testing Strategy

Tier	Status	Coverage
Unit	**Implemented**	JUnit 5 and Mockito over the pricing strategies, parcel state machine and webhook retry policy
Integration	**Implemented**	Testcontainers providing real PostgreSQL and RabbitMQ for repository and messaging tests
Authorisation	**Implemented**	`RbacAuthorizationIntegrationTest`, `DispatchAuthorizationTest` and `PricingAuthorizationTest` assert that unauthorised roles are refused and that spoofed identity headers are rejected
Concurrency	**Implemented**	`AssignmentConcurrencyTest` proves capacity limits and unique-assignment protection against a real database
Contract	**Implemented**	Golden-file tests pin every event's JSON shape in the shared contracts module
API	**Implemented**	Spring MockMvc and WebTestClient slices
End-to-end browser	**Planned**	Playwright smoke tests are specified but not present in the web application manifest

Table 10.1: *Testing tiers and status*

10.2 Defect Audit

Verifying this report against the implementation surfaced six defects. They are recorded here because the diagrams in this document depict the system as it actually behaves.

1. **JWT claim name mismatch.** identity-service writes the merchant claim as `merchantId`; parcel-service, the gateway identity filter and tracking-service all read `merchant_id`. Merchant-scoped operations performed with a user token cannot resolve a merchant.
2. **Dead bearer-token branch in merchant order creation.** The controller tests whether the authentication object is a `Jwt`, but it is a `JwtAuthenticationToken` whose *principal* is the `Jwt`. The test never passes, so even a console-originated merchant order falls through to the service-token path. Functionally benign today; incorrect as written.
3. **API-key requests cannot traverse the gateway.** The gateway security chain is JWT-only and has no API-key filter, so a request carrying an API key is refused at the edge. The documented merchant integration therefore works only against merchant-service directly, which is how Figure 6.2 draws it.
4. **Unauthenticated internal identity lookup.** notification-service calls the internal user lookup with no authorisation header. The endpoint requires a scoped service token, so the

call fails and a catch-all fallback substitutes a synthetic placeholder address rather than failing loudly.

5. **Dashboard aggregates endpoints that do not exist.** The gateway BFF endpoint calls a summary route on parcel, dispatch and merchant services. No such route is implemented on any of them, so the dashboard always returns unavailable sections.
6. **Public tracking detail endpoint is not public.** The web application calls a parcel detail-by-tracking route as though anonymous, but parcel-service permits only the summary route without authentication.

10.3 Consolidated Implementation Status

Capability	Status	Remark
Identity, RBAC, JWT, JWKS, Google OAuth PKCE	**Implemented**	Including refresh rotation and replay revocation
Zone-based pricing with Strategy engine	**Implemented**	Temporal rules with overlap protection
Parcel booking, state machine, receipts	**Implemented**	Idempotent booking
Proof-of-delivery capture to object storage	**Implemented**	Presigned retrieval
Courier dispatch, assignment, workload	**Implemented**	Command-based with compare-and-set
Merchant onboarding, API keys, orders	**Implemented**	Scoped keys, hashed at rest
Outbound signed webhooks with retry	**Implemented**	Exponential backoff to one hour
Real-time tracking ladder	**Implemented**	STOMP, SSE, polling
Email notification pipeline	**Implemented**	Preference-aware, idempotent
SMS notification channel	**Partial**	Mock provider only
AI assistant read tools	**Implemented**	Three tools
AI assistant write/automation tools	**Planned**	Booking, dispatch, merchant operations
MCP server exposure	**Partial**	Dependency declared, wiring absent
Volumetric dimension pricing	**Planned**	Size bands today
Merchant volume discounts	**Planned**	No discount logic
Recipient-addressed notifications	**Planned**	Owner-addressed today
Docker Compose local orchestration	**Implemented**	Full stack
Playwright end-to-end tests	**Planned**	Not in the manifest

Table 10.2: *Consolidated implementation status matrix*

Chapter 11

Team Contributions and Project Management

11.1 Work Breakdown

Index	Member	Area	Key contributions
234021D	Arangala A. P.	Identity & Auth	RS256 token service, JWKS endpoint, Google OAuth PKCE, refresh rotation, RBAC test suite
	Kevin Gaveshana	Gateway & Edge	Spring Cloud Gateway routes, Redis rate limiting, circuit breakers, Docker Compose infrastructure
234127H	M.A.P.M. Mallawasekara	Pricing & Zones	Strategy chain, zone and city resolution, temporal pricing rules
234087H	L. P. Ishadi	Parcel Lifecycle	Parcel state machine, tracking number generation, receipt generation, transactional outbox
234149C	Methsara	Dispatch & Couriers	Courier roster, workload accounting, assignment commands, proof-of-delivery upload
	Charithma Dassanayake	Merchant API	API key lifecycle, order ingestion, HMAC-signed webhooks with retry and dead-lettering
234009A	Maheesha Paboth	Live Tracking	STOMP over SockJS, SSE fallback, Redis projection, public tracking timeline
234208H	Thranjaya S. J. K.	Notifications	AMQP consumer, Mailpit integration, Thymeleaf templates, preference engine, PII masking
234234H	R.K.T.P. Wijerathna	AI Assistant	Spring AI integration, tool callbacks, SSE streaming of tool invocations
234035A	K.G.T. Buddhima	Frontend & Design	React SPA for all four personas, realtime transport ladder, verified architecture documentation and the eighteen diagrams in this report

Table 11.1: *Ten-member work breakdown*

11.2 Milestone Progression

M1 - Foundation Multi-module Maven aggregator, database schemas, gateway routing, JWKS validation.

M2 - Core flows Registration, Google OAuth, fee calculation, parcel booking.

M3 - Events RabbitMQ pipeline, courier assignment, live tracking, email notification.

M4 - Partner & admin Merchant API keys, order ingestion, webhook delivery, admin consoles.

M5 - Hardening Testcontainers integration tests, RBAC suites, and the verified design documentation on which this report is based.

Chapter 12

Conclusion and Roadmap

12.1 Conclusion

SwiftDrop demonstrates that a microservice architecture with an API gateway, database-per-service isolation, and event-driven integration delivers meaningful decoupling and resilience for a logistics domain. The implementation applies recognised patterns - transactional outbox, idempotent consumers, CQRS projections, Strategy-based pricing, circuit breaking and asymmetric authentication - and backs them with an automated test suite including real-infrastructure integration tests.

The report is deliberate about what is not finished. Two substantial items, browser end to end testing and the write capable AI tooling, remain designed rather than delivered, and six defects found during verification are recorded in Section 10.2. Documenting these alongside the working system is a more useful engineering artefact than a report that claims uniform completeness.

12.2 Roadmap

1. **Close the six recorded defects**, beginning with the JWT claim-name mismatch, which silently disables merchant-scoped operations.
2. **Add Playwright end-to-end smoke tests** covering booking, courier status update and public tracking.
3. **Extend the AI assistant with write-capable tools** - booking, dispatch assignment and merchant operations - behind per-authority tool filtering, and complete the MCP server wiring.
4. **Introduce volumetric pricing and merchant volume discounts** as additional strategies in the existing ordered chain, which requires no change to the pricing engine.
5. **Address the recipient** as well as the sender in the notification pipeline.
6. **Integrate a production SMS provider** behind the existing channel abstraction.
7. **Longer term:** route optimisation, native courier mobile clients with offline sync, and payment gateway integration for merchant billing and cash-on-delivery reconciliation.